

SAE 3.01 – Développement d'applications

HiTask

Application de gestion de tâches

Malik Muhammet Bayram
Loïc Durand
Léo Thomas
Johan Schaeffer

Classe S3D

Table des matières

1	Introduction	3
2	Fonctionnalités réalisées	3
2.1	Gestion des projets	3
2.2	Modèle de tâches	3
2.3	Gestion de l'avancement	3
2.4	Vue Bureau (tableau Kanban)	4
2.5	Vue Liste (agenda hebdomadaire)	4
2.6	Vue Gantt	4
2.7	Interface générale	4
3	Choix d'interface et justification des vues	5
3.1	Vue Bureau : représentation Kanban	5
3.2	Vue Liste : agenda hebdomadaire	5
3.3	Vue Gantt : vision globale dans le temps	5
3.4	Changement de vue	5
4	Patrons de conception et architecture	7
4.1	Patron Observer	7
4.2	Architecture MVC	7
4.3	Patron Composite	7
5	Éléments modifiés par rapport à l'étude préalable	8
6	Diagrammes de classes	9
6.1	Diagramme de classe de l'étude préalable	9
6.2	Diagramme de classe final	9
6.3	Comparaison et évolutions	10
7	Répartition du travail entre les étudiants	11
8	Présentation d'un élément dont nous sommes fiers	12
8.1	Johan Schaeffer	12
8.2	Malik Muhammet Bayram	14
8.3	Loïc Durand	15
8.4	Léo Thomas	17
9	Mode d'emploi pour lancer l'application sous IntelliJ	19
9.1	Pré-requis	19

9.2	Organisation du projet	19
9.3	Import du projet dans IntelliJ	19
9.4	Lancement depuis IntelliJ	19
9.5	Accès aux fonctionnalités	19

1 Introduction

Dans le cadre de la SAE 3.01 « Développement d'applications », nous devons réaliser une application JavaFX de gestion de tâches pour un projet. L'application devait proposer plusieurs façons d'afficher les tâches, permettre de sauvegarder les données dans un fichier et utiliser des patrons de conception vus en cours.

Nous avons développé *HiTask*, une application qui aide à organiser et suivre les tâches d'un projet. Elle permet notamment :

- de créer des listes de tâches et de les organiser selon leur état ;
- de définir des tâches avec des dates, une priorité, des sous-tâches et des dépendances ;
- de visualiser le projet sous trois formes : un tableau Kanban, un agenda hebdomadaire et un diagramme de Gantt.

Dans ce rapport, nous présentons les principales fonctionnalités de l'application, les choix d'interface (en particulier pour les différentes vues), ainsi que les patrons de conception et l'architecture que nous avons mis en place.

2 Fonctionnalités réalisées

2.1 Gestion des projets

- Création d'un nouveau projet sous la forme d'un fichier `.htsk` à partir de la page d'accueil.
- Ouverture d'un projet existant et chargement de toutes les données associées.
- Sauvegarde du projet après les opérations importantes (ajout, suppression, modification ou déplacement) afin de limiter les risques de perte de données.
- Mémoire de la dernière vue utilisée afin de retrouver le même écran à la réouverture du projet.
- Début d'une fonctionnalité d'archivage pour préparer la gestion de projets anciens.

2.2 Modèle de tâches

- Gestion de listes de tâches (colonnes) représentant différents états du travail, par exemple « À faire », « En cours », « Terminé ».
- Création, modification et suppression de tâches avec : titre, description, dates de début et de fin, priorité (Normal, Important, Urgent).
- Vérification simple de la cohérence des dates (début avant fin) et cohérence entre la tâche parente et ses sous-tâches.
- Possibilité de marquer une tâche comme effectuée grâce à une case à cocher.
- Gestion des sous-tâches grâce à une hiérarchie de tâches simples et de tâches composites.
- Gestion de dépendances entre tâches : une tâche peut nécessiter qu'une autre soit terminée avant de commencer.

2.3 Gestion de l'avancement

- Propagation de l'état « fait / non fait » : cocher ou décocher une tâche met à jour, si besoin, l'état de la tâche parente ou de ses sous-tâches.
- Mise à jour automatique de toutes les vues lorsque le modèle change (ajout, suppression, changement d'état, déplacement, etc.).

2.4 Vue Bureau (tableau Kanban)

- Affichage des listes sous forme de colonnes alignées horizontalement.
- Affichage des tâches sous forme de cartes contenant le titre, la priorité, les dates et la description.
- Affichage des sous-tâches directement dans la carte de la tâche parente.
- Glisser-déposer des cartes pour déplacer une tâche d'une liste à une autre.
- Glisser-déposer d'une carte sur une autre pour transformer une tâche en tâche composite et créer une relation de sous-tâche.
- Boutons pour créer une tâche, créer une sous-tâche et supprimer une tâche ou une liste.

2.5 Vue Liste (agenda hebdomadaire)

- Affichage des tâches sur une semaine, organisée par jour.
- Une tâche apparaît pour chaque jour où elle est active (entre sa date de début et sa date de fin), ce qui permet de voir la charge quotidienne.
- Tri possible des tâches par titre, priorité ou durée.
- Filtre possible sur l'état des tâches (toutes, seulement faites, seulement non faites).
- Affichage des sous-tâches avec un léger décalage visuel pour montrer la hiérarchie.
- Affichage d'une ligne « Après : ... » lorsque la tâche a une tâche préalable.

2.6 Vue Gantt

- Représentation des tâches sous forme de barres sur une frise temporelle.
- Affichage des dates sur l'axe horizontal pour visualiser la durée et les chevauchements des tâches.
- Liste de tâches avec cases à cocher à gauche pour choisir quelles tâches afficher dans le diagramme.
- Boutons de zoom et bouton pour revenir rapidement à la date du jour.

2.7 Interface générale

- Page d'accueil permettant de créer un nouveau projet ou d'ouvrir un projet existant.
- Barre de menu supérieure affichant le nom du fichier de projet courant.
- Boutons de navigation pour passer facilement de la vue Bureau à la vue Liste, à la vue Gantt ou à la page d'accueil.
- Utilisation d'icônes et de styles homogènes pour les actions principales (création, suppression, navigation).

3 Choix d'interface et justification des vues

3.1 Vue Bureau : représentation Kanban

Pour la vue Bureau, nous avons choisi un tableau de type Kanban avec des colonnes « À faire », « En cours » et « Terminé ». Ce type de présentation est simple à comprendre et déjà connu dans de nombreux outils de gestion de projet. L'utilisateur voit tout de suite où en sont ses tâches. Le glisser-déposer entre les colonnes permet de changer l'état d'une tâche sans passer par une fenêtre de paramètres, ce qui rend l'application plus agréable à utiliser. Nous avons aussi décidé d'afficher les sous-tâches directement dans la carte de la tâche parente pour que toutes les informations restent regroupées. La vue Bureau répond donc principalement à la question : « Où en sont mes tâches dans le flux de travail ? ».

3.2 Vue Liste : agenda hebdomadaire

La vue Liste a été conçue pour aider à organiser le travail au jour le jour. Au lieu de classer les tâches par état, nous les affichons en fonction des jours de la semaine. Une tâche qui dure plusieurs jours est répétée sur chaque jour où elle est active, ce qui permet de voir d'un coup d'œil ce qui est prévu aujourd'hui, demain, etc. Nous avons ajouté des options de tri (par titre, priorité, durée) et de filtre (toutes, faites, non faites) pour que l'utilisateur puisse adapter l'affichage à ses besoins : préparer sa journée, se concentrer sur les tâches urgentes, ou voir uniquement ce qui reste à faire. Les sous-tâches sont affichées en dessous de la tâche parente avec un petit décalage, ce qui montre la hiérarchie tout en restant lisible. Cette vue répond à la question : « Qu'est-ce que j'ai à faire cette semaine, jour par jour ? ».

3.3 Vue Gantt : vision globale dans le temps

La vue Gantt offre une vision plus globale du projet dans le temps. Les vues Bureau et Liste sont pratiques pour le suivi quotidien, mais elles ne montrent pas facilement les durées et les chevauchements des tâches sur plusieurs jours ou semaines. Avec le diagramme de Gantt, chaque tâche est représentée par une barre qui s'étend de sa date de début à sa date de fin. L'utilisateur peut ainsi repérer rapidement les périodes chargées, les tâches qui se déroulent en parallèle et les éventuels « trous » dans le planning. La liste de tâches avec cases à cocher permet de masquer certaines tâches pour simplifier le diagramme lorsque le projet devient plus grand. Les boutons de zoom et le recentrage sur la date du jour facilitent la navigation sur la frise temporelle. Cette vue répond à la question : « Comment se place mon projet dans le temps ? ».

3.4 Changement de vue

La navigation entre les différentes vues de l'application est gérée depuis `VueMenu`, qui contient quatre boutons permettant d'afficher respectivement : la page d'accueil, la vue Bureau, la vue Liste et la vue Gantt.

Une seule vue est affichée à la fois. Pour cela, nous utilisons les propriétés `setVisible()` et `setManaged()` :

- la vue active est mise en `visible = true` et `managed = true`;
- toutes les autres vues sont désactivées via `visible = false` et `managed = false`.

Le changement de vue est centralisé dans la méthode `switchView()`, ci-dessous :

Listing 1 – Méthode de changement de vue

```

public void switchView(String type) {
    homePage.setVisible(false);
    homePage.setManaged(false);
    vb.setVisible(false);
    vb.setManaged(false);
    vl.setVisible(false);
    vl.setManaged(false);
    vg.setVisible(false);
    vg.setManaged(false);

    switch (type) {
        case "HOME" -> { homePage.setVisible(true); homePage.setManaged(true); }
        case "BUREAU" -> { vb.setVisible(true); vb.setManaged(true); }
        case "LISTE" -> { vl.setVisible(true); vl.setManaged(true); }
        case "GANTT" -> { vg.setVisible(true); vg.setManaged(true); }
    }
}

```

Cette méthode est appelée par le **ControlerMenu**, auquel sont reliés les quatre boutons de navigation via **setOnAction**. Ainsi, chaque clic déclenche l'affichage de la vue correspondante tout en masquant les autres.

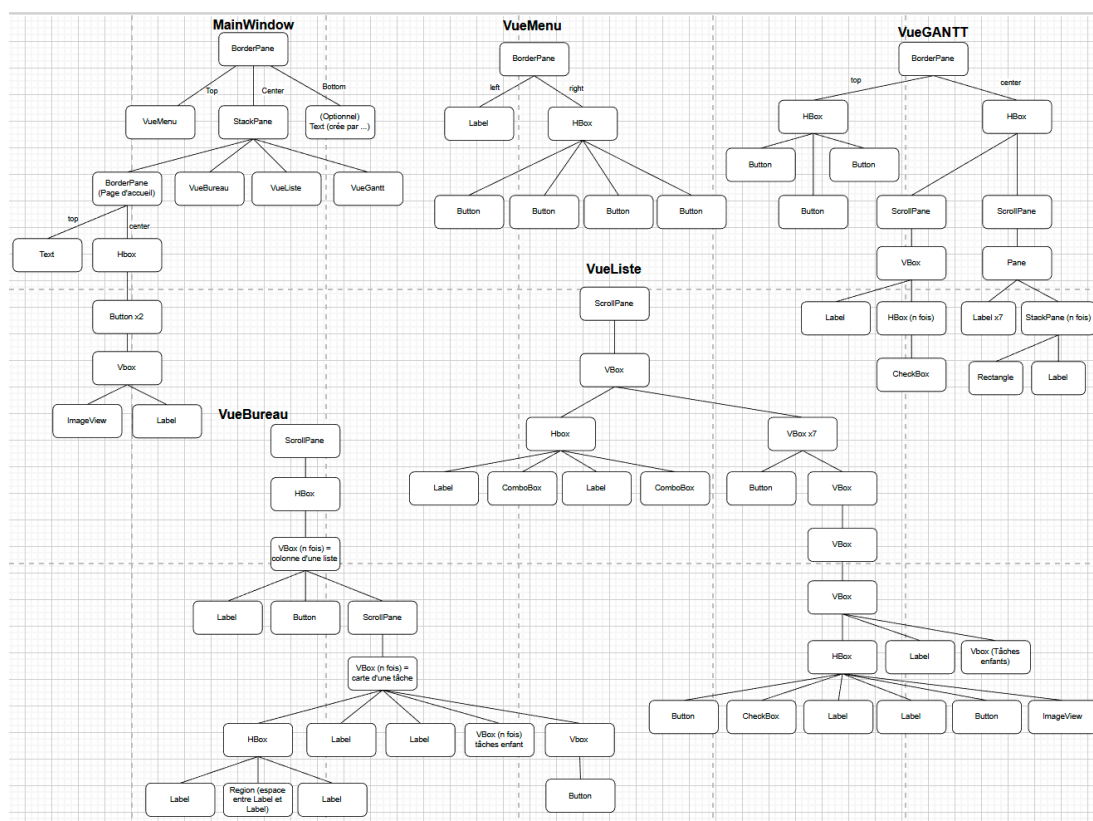


FIGURE 1 – Graphe de scène regroupant toutes les vues et la MainWindow

4 Patrons de conception et architecture

Dans l'application HiTask, nous avons utilisé plusieurs patrons de conception afin d'avoir un code plus clair et plus facile à faire évoluer. Les principaux sont les patrons **Observer**, **MVC** et **Composite**.

4.1 Patron Observer

Le modèle implémente une interface **Sujet** et maintient une liste d'objets **Observateur**. Les différentes vues graphiques (**VueBureau**, **VueListe**, **VueGantt**, **VueMenu**, etc.) implémentent toutes l'interface **Observateur** et s'enregistrent auprès du modèle. À chaque modification importante des données (ajout ou suppression de tâches, changement d'état, déplacement, modification d'une liste), le modèle appelle `notifierObservateur()`, ce qui déclenche la mise à jour de toutes les vues. Ce fonctionnement correspond au patron *Observer* : un sujet central prévient automatiquement plusieurs observateurs lorsque son état change, sans dépendre directement d'eux.

4.2 Architecture MVC

L'application suit une architecture de type *Model-View-Controller* adaptée à JavaFX.

- Le **Model** est constitué de la classe **Model** et des classes métier **Liste**, **Tache**, **CompositeTache** et **FeuilleTache**. Elles stockent les données (titres, dates, priorités, liens entre tâches) et contiennent la logique de base : ajout, suppression, déplacement, calcul de l'état, etc.
- Les **Views** sont les classes **MainWindow**, **VueMenu**, **VueBureau**, **VueListe** et **VueGantt**. Elles s'occupent uniquement de l'affichage et de la mise en forme des données (tableau Kanban, agenda, Gantt).
- Les **Controllers** (**ControllerMenu**, **ControllerHomeBtn**, **ControllerDrag**, **ControllerPopListe**, **ControllerPopTache**, etc.) récupèrent les actions de l'utilisateur (clics, drag & drop, validation de formulaires) et appellent les méthodes du modèle. Ils demandent ensuite aux vues de se mettre à jour.

Cette séparation entre modèle, vues et contrôleurs rend le code plus lisible, plus testable et plus simple à modifier.

4.3 Patron Composite

La gestion des tâches et des sous-tâches s'appuie sur le patron *Composite*. La classe abstraite **Tache** représente une tâche de manière générale. La classe **FeuilleTache** hérite de **Tache** et correspond à une tâche simple, sans sous-tâches. La classe **CompositeTache** hérite aussi de **Tache**, mais contient en plus une liste de **Tache** qui représente ses sous-tâches.

Grâce à ce choix, le code peut manipuler une tâche simple ou une tâche composite presque de la même manière. Les vues ne sont pas obligées de connaître le type précis : si l'objet est une **CompositeTache**, elles parcourent simplement sa liste d'enfants pour les afficher. Ce patron permet donc de gérer proprement la hiérarchie des tâches tout en gardant une interface de programmation simple.

5 Éléments modifiés par rapport à l'étude préalable

Lors de la phase d'analyse, nous avons produit des maquettes d'écrans, un premier diagramme de classes et plusieurs diagrammes de séquence décrivant les principaux scénarios d'utilisation. L'application finale reste globalement fidèle à cette base, mais certains éléments ont été ajustés au fur et à mesure du développement.

Sur la partie interface, les maquettes ont été respectées dans leurs grandes lignes : on retrouve la page d'accueil, la vue Bureau, la vue Liste et la vue Gantt. En revanche, nous avons simplifié et déplacé quelques éléments. Par exemple, certains boutons ont été regroupés ou renommés pour être plus clairs. Nous avons aussi ajouté des informations qui n'apparaissaient pas toujours dans les premiers écrans, comme un badge de priorité plus visible ou un texte explicite pour les dépendances entre tâches.

Un autre choix important a été d'afficher les sous-tâches directement à l'intérieur des cartes de la vue Bureau et dans la vue Liste. Ce point n'était pas forcément détaillé dans les maquettes d'origine, mais il est apparu comme plus pratique après les premiers essais, car il évite à l'utilisateur d'ouvrir plusieurs fenêtres pour comprendre la structure d'une tâche.

Le diagramme de classes issu de l'analyse a lui aussi évolué. La structure générale (présence d'un modèle central, de listes et de tâches) a été conservée, mais plusieurs détails ont été modifiés. Nous avons notamment affiné la hiérarchie autour de **Tache** en séparant clairement les tâches simples (**FeuilleTache**) et les tâches composites (**CompositeTache**), et en ajoutant des attributs liés aux dépendances et à la gestion de l'état « fait / non fait ».

La classe **Model** a également été enrichie par rapport à la version de l'analyse. Elle gère maintenant les déplacements de tâches (entre listes ou sous une tâche composite), la notification des vues via le patron Observer, ainsi que la persistance des données dans les fichiers **.htsk**. Certains diagrammes de séquence ont été ajustés en conséquence pour mieux refléter la répartition réelle entre modèle, vues et contrôleurs.

Au final, l'étude préalable nous a servi de base solide, mais nous avons accepté de modifier certains éléments (interface, diagramme de classes, scénarios) pour nous adapter aux contraintes techniques et aux retours obtenus pendant le développement.

6 Diagrammes de classes

6.1 Diagramme de classe de l'étude préalable

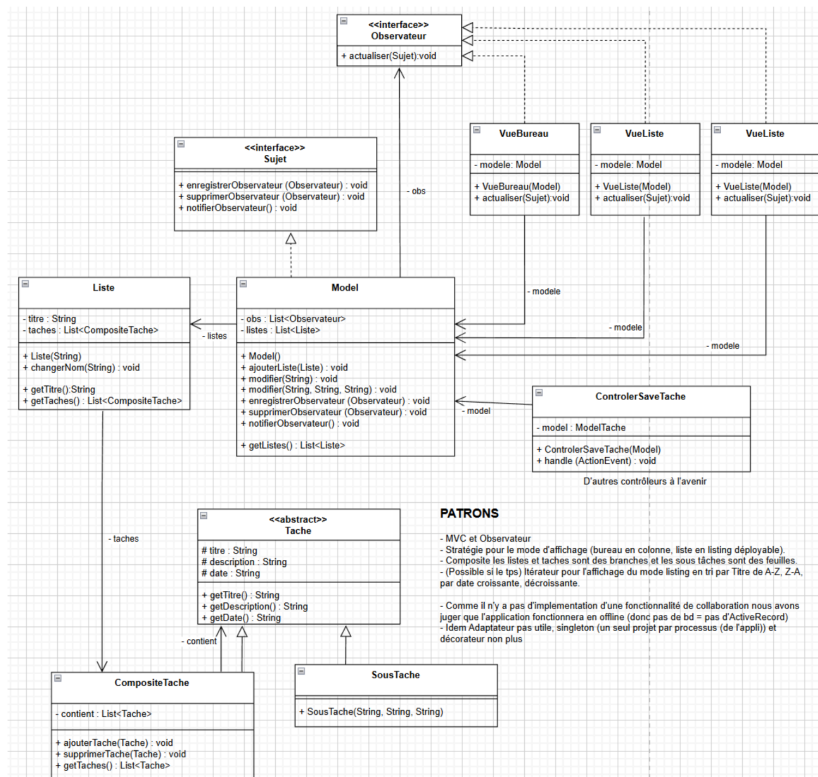


FIGURE 2 – Diagramme de classe issu de la phase d'analyse

6.2 Diagramme de classe final

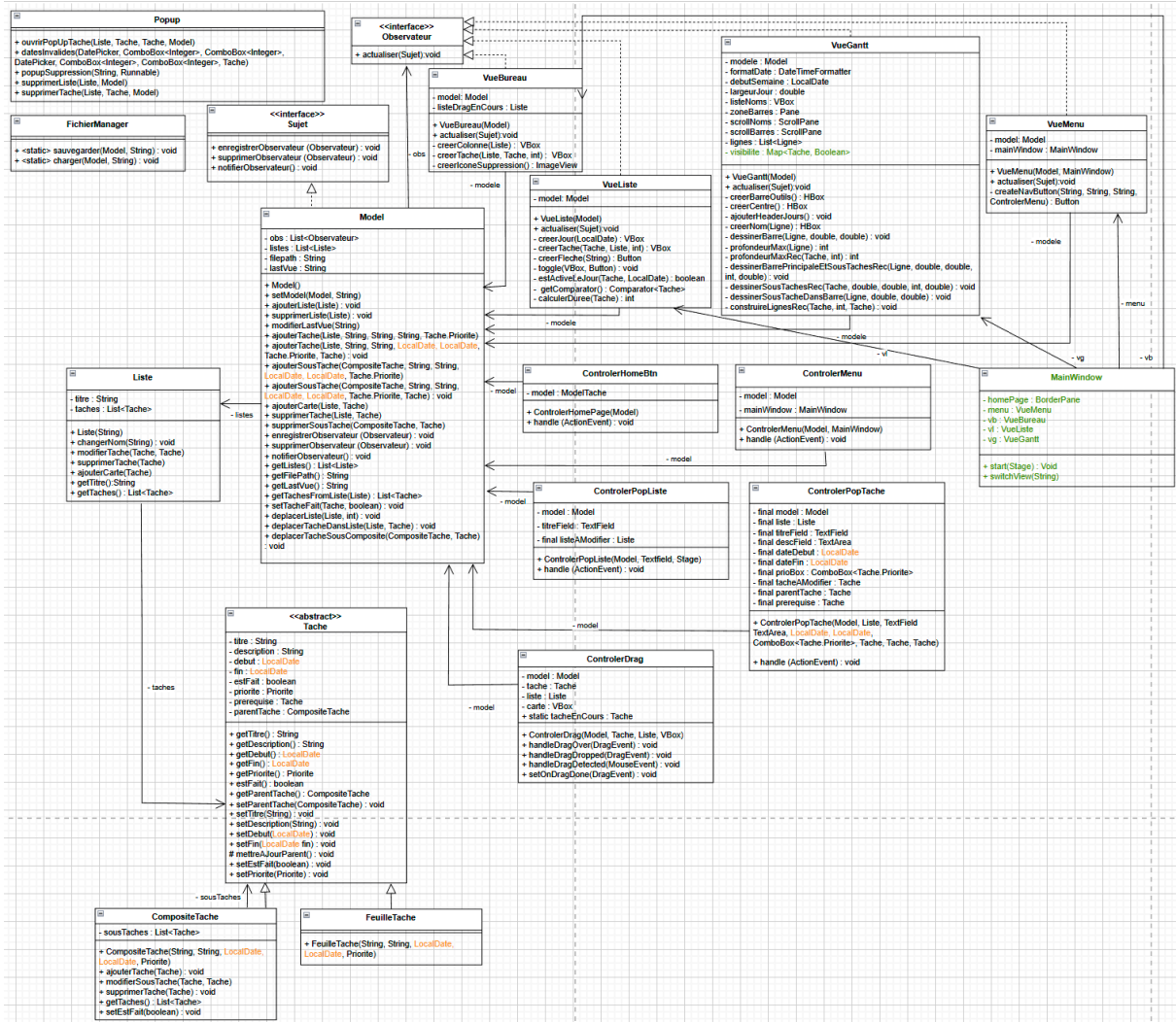


FIGURE 3 – Diagramme de classe final de l'application HiTask

6.3 Comparaison et évolutions

Le diagramme de classe de l'étude préalable représente déjà les éléments principaux du domaine : les listes, les tâches et les sous-tâches. Il pose la base de la hiérarchie entre **Tache** et **SousTache**, ainsi que l'idée d'un modèle central chargé de gérer les listes.

Dans le diagramme final, cette structure a été conservée mais nettement détaillée. La classe **Tache** a été raffinée en deux sous-classes distinctes, **FeuilleTache** et **CompositeTache**, afin d'appliquer clairement le patron Composite. Le modèle **Model** a été enrichi avec les opérations nécessaires à l'application réelle (ajout et suppression de tâches, gestion des sous-tâches, déplacements, sauvegarde des données, etc.).

On voit également apparaître l'ensemble des vues JavaFX (**VueBureau**, **VueListe**, **VueGantt**, **VueMenu**) et des contrôleurs associés. Cela traduit le passage d'une vision assez abstraite lors de l'analyse à une architecture complète de type MVC/Observer dans l'implémentation finale. En résumé, le diagramme final reste fidèle aux idées de départ, mais précise les responsabilités de chaque classe et ajoute tout ce qui était nécessaire pour faire fonctionner réellement l'application.

7 Répartition du travail entre les étudiants

Afin d'assurer une progression du projet, nous avons réparti les tâches de manière équitable entre les membres du groupe au préalable durant la partie analyse. Nous nous entraînions régulièrement lorsque l'un d'entre nous rencontrait une difficulté, ce qui a permis de maintenir une bonne cohérence globale du projet et respecter au mieux le planning initial.

Itération 1

- Mise en place projet JavaFX + MVC (Johan, Muhammet, Loïc)
- Liste + TestListe (Loïc)
- Patron Composite (Léo)
- Affichage Terminal (Muhammet)
- Page d'accueil + Vue Menu (début, non terminé)
- Vue Liste (début, non terminé)

Itération 2

- Vue Menu (Léo)
- Fonctionnement des tâches (Malik, Loïc, Johan)
- Chargement fichier (Malik, Léo)
- Vue Bureau (début, non terminé)

Itération 3

- Création + modification tâche et liste (Johan)
- Vue Bureau finalisée (Léo, Johan)
- Déplacement tâche/liste (non terminé)

Itération 4

- Drag & Drop
- Sous-tâches + gestion dates/heures
- Vue Liste
- Tests

Itération 5

- Cohérence dates/heures tâche sous-tâche
- Correction VueListe
- Dépendances fonctionnelles + suppression heure :minute
- Diagramme de Gantt (début)
- Archivage (début)

Itération 6

- Commentaires + optimisation (Johan, Léo)
- Vue Archive (Johan)
- Améliorations ergonomiques Vue Bureau (Johan)
- Archivage (non terminé)
- Diagramme de Gantt (Loïc)

8 Présentation d'un élément dont nous sommes fiers

8.1 Johan Schaeffer

L'élément dont je suis le plus fier dans ce projet est la fenêtre popup qui permet de **créer et modifier** les tâches (et sous-tâches). Cette fenêtre regroupe plusieurs aspects importants : la saisie des informations de base (titre, description, dates, priorité), la gestion des sous-tâches, le choix d'une tâche préalable et la vérification des dates pour éviter les erreurs.

Mon objectif était d'avoir une seule méthode capable de gérer tous les cas : création d'une nouvelle tâche, création d'une sous-tâche et modification d'une tâche existante. Pour cela, la méthode `ouvrirPopUpTache` reçoit la liste cible, la tâche à modifier (ou `null` si l'on crée une nouvelle tâche), la tâche parente éventuelle et le modèle. La popup adapte automatiquement son titre (« Créer une tâche » ou « Modifier une tâche ») et préremplit les champs quand on est en mode édition.

Listing 2 – Extrait de la méthode `ouvrirPopUpTache`

```
public static void ouvrirPopUpTache(Liste liste,
                                   Tache tacheAModifier,
                                   Tache parentTache,
                                   Model model) {

    boolean modeModification = (tacheAModifier != null);

    Dialog<ButtonType> dialog = new Dialog<>();
    dialog.setTitle(modeModification ? "Modifier une tche"
                                     : "Crer une tche");

    TextField champTitre = new TextField(
        modeModification ? tacheAModifier.getTitre() : ""
    );
    champTitre.setPromptText("Nom de la tche");

    TextArea champDescription = new TextArea(
        modeModification
            ? (tacheAModifier.getDescription() == null ? ""
               : tacheAModifier.getDescription())
            : ""
    );
    champDescription.setPromptText("Description");
}
```

La logique de vérification des dates s'appuie sur un `Runnable verifierDates`. L'idée du `Runnable` vient de Léo; de mon côté, je l'ai intégré dans la popup et relié à la méthode `datesInvalides` pour contrôler la cohérence des dates et l'état du bouton « Enregistrer ». Les champs de début et

de fin sont initialisés différemment selon le contexte (création simple, sous-tâche, modification). Ensuite, le **Runnable** est appelé à chaque changement de valeur.

Listing 3 – Validation des dates dans la popup

```
Runnable verifierDates = () -> {
    boolean invalide = datesInvalides(dateDebut, dateFin, parentTache);
    boolean titreVide = champTitre.getText() == null
        || champTitre.getText().isBlank();

    bValider.setDisable(invalide || titreVide);
    labErreurDate.setText(invalide ? "Date invalide" : "");
};
```

Un autre point sur lequel j’ai travaillé est la gestion des dépendances. La popup propose une `ComboBox` qui contient toutes les tâches et sous-tâches possibles, en évitant de proposer la tâche en cours de modification. Pour remplir cette liste, j’ai écrit une petite méthode récursive `ajouterTacheEtSousTachesDansCombo` qui parcourt tout l’arbre des tâches.

Listing 4 – Méthode récursive pour remplir la liste des dépendances

```
private static void ajouterTacheEtSousTachesDansCombo(ComboBox<Tache> combo,
                                                    Tache t,
                                                    Tache tacheAModifier) {

    if (t != tacheAModifier) {
        combo.getItems().add(t);
    }
    if (t instanceof CompositeTache ct) {
        for (Tache sous : ct.getTaches()) {
            ajouterTacheEtSousTachesDansCombo(combo, sous, tacheAModifier);
        }
    }
}
```

Au final, cette popup regroupe dans une seule interface la création et la modification des tâches, la gestion des sous-tâches et des dépendances, tout en garantissant la cohérence des dates. C’est une partie du code que je trouve à la fois utile pour l’utilisateur et intéressante à avoir mise en place.

8.2 Malik Muhammet Bayram

La fonctionnalité dont je suis la plus fière est le *drag and drop* d’une tâche vers une liste. Cela paraissait très compliqué au départ, mais finalement c’était plutôt facile grâce à l’exemple donné sur Arche. La conception en MVC rendait son implémentation plus simple et plus logique.

J’ai donc créé un `ControlerDrag` avec quatre handlers. Voici un exemple de handler implémenté :

```
public void handleDragDetected(MouseEvent e) {
    tacheEnCours = tache;
    Dragboard db = carte.startDragAndDrop(TransferMode.MOVE);

    ClipboardContent content = new ClipboardContent();
    content.putString("TACHE");
    db.setContent(content);

    e.consume();
}
```

Ce handler sert à détecter la tâche dans la vue principale (*main*).

```
public void handleDragOver(DragEvent e) {
```

```

    if (ControlerDrag.tacheEnCours != null) {
        e.acceptTransferModes(TransferMode.MOVE);
    }
    e.consume();
}

```

Ce handler permet d'indiquer que la zone peut accepter un dépôt lorsqu'une tâche est en cours de déplacement.

Dans la classe `VueBureau`, j'ai ajouté des écouteurs simples :

```

ControlerDrag cd = new ControlerDrag(model, tsk, liste, carte);
carte.setOnDragDetected(cd::handleDragDetected);
carte.setOnDragOver(cd::handleDragOver);
carte.setOnDragDropped(cd::handleDragDropped);
carte.setOnDragDone(cd::setOnDragDone);

```

Je n'ai pas implémenté le déplacement d'une tâche vers une sous-tâche, car la notion de sous-tâche n'était pas encore présente à ce stade du projet. Cependant, le fait d'avoir mis en place ce mécanisme a facilité par la suite l'implémentation du *drag and drop* des listes.

8.3 Loïc Durand

L'élément du projet dont je suis le plus fier est l'affichage des tâches et des sous-tâches dans le diagramme de Gantt. L'objectif était de représenter visuellement la hiérarchie des tâches tout en conservant une lecture claire et intuitive pour l'utilisateur. Chaque tâche principale est affichée sous forme d'une barre bleue, et ses sous-tâches apparaissent à l'intérieur de cette barre sous forme de rectangles verts, éventuellement imbriqués sur plusieurs niveaux.

L'un des enjeux principaux était de gérer correctement les sous-tâches de sous-tâches. Pour cela, j'ai mis en place une logique récursive qui parcourt l'arbre des tâches et permet de dessiner chaque niveau à une position verticale différente à l'intérieur de la tâche principale. La hauteur de la barre principale est alors calculée dynamiquement en fonction de la profondeur maximale de ses sous-tâches.

J'ai créé une classe interne `Ligne` qui représente une tâche à afficher (nom, niveau, dates, parent, visibilité). La méthode `construireLignesRec` parcourt récursivement les tâches et sous-tâches afin de conserver la hiérarchie.

Listing 5 – Construction récursive des lignes du Gantt

```
private void construireLignesRec(Tache t, int niveau, Tache parent) {
    visibilite.putIfAbsent(t, true);

    Ligne ligne = new Ligne(
        t.getTitre(),
        niveau,
        t.getDebut(),
        t.getFin(),
        t,
        parent
    );

    ligne.visible = visibilite.get(t);
    lignes.add(ligne);

    if (t instanceof CompositeTache ct) {
        for (Tache st : ct.getTaches()) {
            construireLignesRec(st, niveau + 1, t);
        }
    }
}
```

Cette méthode permet de gérer un grand nombre de niveaux de sous-tâches, sans limitation sur la profondeur.

Pour que toutes les sous-tâches soient visibles à l'intérieur d'une tâche principale, la hauteur de la barre bleue doit s'adapter automatiquement. J'ai donc calculé la profondeur maximale des sous-tâches visibles et utilisé cette information pour déterminer la hauteur totale de la barre.

Listing 6 – Calcul de la profondeur maximale des sous-tâches

```
private int profondeurMax(Ligne racine) {
    return profondeurMaxRec(racine.tacheAssociee, 0);
}

private int profondeurMaxRec(Tache tache, int niveauActuel) {
    int max = niveauActuel;
    if (tache instanceof CompositeTache ct) {
        for (Tache st : ct.getTaches()) {
            for (Ligne l : lignes) {
                if (l.tacheAssociee == st && l.visible) {
                    max = Math.max(max,
                        profondeurMaxRec(st, niveauActuel + 1));
                    break;
                }
            }
        }
    }
    return max;
}
```

Grâce à cela, la barre principale s'agrandit automatiquement lorsque de nouvelles sous-tâches sont ajoutées ou rendues visibles.

Pour améliorer la lisibilité, le nom de chaque tâche est affiché directement dans sa barre. Lorsque

la tâche principale possède des sous-tâches, le texte est placé en haut de la barre afin de ne pas être masqué par les rectangles verts. Chaque barre (principale ou sous-tâche) est cliquable et ouvre une popup (présentée par Johan juste avant) permettant de consulter ou modifier la tâche correspondante.

8.4 Léo Thomas

L'élément du projet dont je suis le plus fier est la fonctionnalité d'affichage des sous-tâches, ainsi que la gestion de l'état faite / non faite et des dates début/fin. Cette partie a demandé de prendre en compte de nombreuses conditions et paramètres afin de rester parfaitement cohérent avec le patron Composite. L'un des principaux défis a été de réaliser un affichage capable de gérer une arborescence de sous-tâches infinie, tout en restant lisible. La solution retenue consiste à afficher les sous-tâches directement à l'intérieur de leur tâche parente, avec un décalage à gauche dépendant de la variable "profondeur". J'ai aussi choisi de n'afficher le bouton de création de sous-tâche qu'au survol, afin de ne pas surcharger l'interface.

Listing 7 – Morceaux de code correspondant à la création de l'affichage des sous tâches

```
private VBox creerTache(Liste liste, Tache tsk, int profondeur) {
    VBox carte = new VBox(6);
    carte.setStyle("-fx-background-color: " + (profondeur % 2 == 0 ? "#ffffff" : "#f3f4f6") + "; -fx-padding: 10px; -fx-background-radius: 10px; -fx-border-color: #e5e7eb; -fx-border-radius: 10px;");
    ...
    VBox sousTachesBox = new VBox(6);
    if (tsk instanceof CompositeTache ct) {
        for (Tache sousTache : ct.getTaches()) {
            // Appel récursif pour cree les sous-tches
            VBox sousCarte = creerTache(liste, sousTache, profondeur + 1);
            sousCarte.setTranslateX(profondeur * 2);
            sousTachesBox.getChildren().add(sousCarte);
        }
    }
    ...
}
```

Un autre défi, en dehors de la partie front-end, a été d'assurer la cohérence logique, notamment pour la gestion de l'état faite/non faite des tâches, ainsi que la cohérence des dates entre les tâches parentes et leurs sous-tâches.

Il était essentiel de garantir que la date de début d'une sous-tâche ne puisse pas être antérieure à celle de sa tâche parente et de même pour la date de fin afin de garder une arborescence temporelle cohérente.

La gestion du statut fait/non fait a également demandé une réflexion pour rester compatible avec le patron Composite. Ces règles ont alors été définies : - Si je coche un parent alors on coche tout les enfants - Si je décoche un parent alors je ne décoche que le parent - Si je coche tous les enfants alors je code le parent - Si au moins un enfant est décoché alors je décoche le parent

Pour implémenter ces règles, plusieurs ajustements ont été nécessaires : Chaque sous-tâche doit stocker une référence vers son parent, afin de remonter dans l'arborescence. Une méthode dédiée, `mettreAJourParent()`, a été ajoutée dans `Tache` pour éviter les boucles infinies lors des appels récursifs de `setEstFait(boolean fait)`.

Listing 8 – Extrait de code de Tache et CompositeTache

```

// Extrait de Tache
public void setEstFait(boolean fait) {
    boolean ancien = this.estFait;
    this.estFait = fait;
    if (ancien == fait) return;
    mettreAJourParent();
}

protected void mettreAJourParent() {
    CompositeTache parent = getParentTache();
    if (parent == null) return;

    if (!this.estFait) {
        if (parent.estFait()) parent.setEstFait(false);
        return;
    }

    boolean tousFaits = parent.getTaches().stream().allMatch(Tache::estFait);
    if (tousFaits && !parent.estFait()) parent.setEstFait(true);
}

//Extrait de CompositeTache
@Override
public void setEstFait(boolean fait) {
    boolean ancien = estFait();
    super.setEstFait(fait);

    if (ancien == fait) return;

    if (getParentTache() != null) {
        for (Tache t : sousTaches) if (t.estFait() != fait) t.setEstFait(fait);
        return;
    }

    if (fait) for (Tache t : sousTaches) if (!t.estFait()) t.setEstFait(true);
}

```

9 Mode d'emploi pour lancer l'application sous IntelliJ

9.1 Pré-requis

- IntelliJ IDEA (Community ou Ultimate), version récente (2023.x ou ultérieure).
- JDK 21 (le projet est compilé en `source` et `target` 21 dans le `pom.xml`).
- Maven (intégré par défaut dans IntelliJ).

Toutes les dépendances (JavaFX et JUnit) sont gérées par Maven dans le fichier `pom.xml`, il n'y a donc pas de `.jar` à ajouter manuellement.

9.2 Organisation du projet

- `src/main/java/appSAE` : code source principal (modèle, vues, contrôleurs).
- `src/test/java` : classes de tests JUnit éventuelles.
- `pom.xml` : configuration Maven, dépendances JavaFX 21.0.6 (`javafx-controls`, `javafx-fxml`) et plugin `javafx-maven-plugin`.

La classe principale de l'application est `appSAE.MainWindow`, déclarée dans la configuration du plugin JavaFX.

9.3 Import du projet dans IntelliJ

1. Cloner le dépôt Git ou télécharger le projet.
2. Dans IntelliJ, choisir *File* → *Open* et sélectionner le dossier contenant le fichier `pom.xml`.
3. Laisser IntelliJ importer le projet Maven et télécharger les dépendances.
4. Vérifier que le SDK du projet est bien un JDK 21 (*File* → *Project Structure* → *Project SDK*).

9.4 Lancement depuis IntelliJ

1. Ouvrir la fenêtre Maven (onglet *Maven* à droite dans IntelliJ).
2. Dans la section *Plugins* → *javafx*, utiliser la cible `javafx:run` pour lancer l'application. Le plugin `javafx-maven-plugin` se charge de configurer le module-path et les modules JavaFX nécessaires.
3. On peut aussi créer une configuration de type *Maven* avec la commande `clean javafx:run`.

Il est également possible de lancer directement la classe principale :

1. Faire un clic droit sur `appSAE.MainWindow` et choisir *Run 'MainWindow'*.
2. IntelliJ réutilise alors la configuration Maven du projet (JavaFX 21.0.6) pour démarrer l'application.

9.5 Accès aux fonctionnalités

Une fois l'application HiTask lancée :

- L'écran d'accueil permet de créer un nouveau fichier `.htsk` ou d'ouvrir un fichier existant.

- La barre de menu (**VueMenu**) permet de naviguer entre les trois vues : **Bureau** (tableau Kanban), **Liste** (agenda hebdomadaire) et **Gantt** (diagramme temporel des tâches).
- Dans chaque vue, les boutons et popups permettent de créer/modifier des tâches, gérer les sous-tâches et dépendances, et sauvegarder automatiquement le projet.